МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №6

"Автоматизоване тестування, мокінг та аналіз покриття коду"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Опанувати методологію автоматизованого модульного та інтеграційного тестування у мові Python за допомогою фреймворку pytest; реалізувати параметризовані тести, граничні перевірки та перевірки винятків (pytest.raises, pytest.approx); навчитися застосовувати механізми тестової ізоляції (tmp_path, monkeypatch) та фікстури життєвого циклу з yield; освоїти використання тестових дублерів (Mock, AsyncMock, Fake), конфігурацію return_value та side_effect, верифікацію взаємодій; провести вимірювання та аналіз statement і branch coverage за допомогою pytest-cov, дослідити поведінку коду в 7 обов'язкових експериментальних сценаріях та верифікувати мутаційну стійкість набору тестів.

Теоретичні відомості:

Автоматизоване тестування є фундаментальним інженерним компонентом розробки надійного програмного забезпечення на Python. Фреймворк pytest надає декларативний механізм виявлення тестів, лаконічний синтаксис на основі стандартного оператора assert та потужну систему фікстур. Фікстури із ключовим словом yield забезпечують чітке розділення фаз setup і teardown, детерміновано звільняючи ресурси після завершення кожного тесту. Стандартна ізоляція досягається використанням тимчасових каталогів tmp_path та підміни оточення через monkeypatch. Для тестування компонентів із зовнішніми залежностями використовуються тестові дублери: Fake надає спрощену реалізацію збереження стану в пам'яті, тоді як Mock дозволяє контролювати повернені значення (return_value), імітувати аварійні збої (side_effect) та перевіряти протокол викликів (interaction testing). Асинхронні корутини перевіряються за допомогою AsyncMock із обов'язковим await. Аналіз покриття коду за допомогою pytest-cov оцінює як виконання лінійних інструкцій (statement coverage), так і проходження всіх альтернативних логічних розгалужень (branch coverage). При цьому виконання коду не тотожне його верифікації (execution != verification), тому наявність змістовних перевірок (assertions) та мутаційна стійкість є пріоритетними критеріями якості тестового набору.

Виконання роботи

1. Архітектура проєкту, варіант 1 та структура модулів і тестів

Відповідно до індивідуального варіанта №1 я реалізував комплексну систему автоматизованого тестування для проєкту 'Система обліку студентів' (student_manager). Архітектура проєкту побудована за модульним принципом, де виробничий код зосереджено в каталозі src/student_manager/, а тестовий набір чітко структуровано в каталозі tests/ із розділенням на модульні тести поведінок (tests/unit/), інтеграційні конвеєри (tests/integration/), асинхронні перевірки (tests/test_async_service.py) та спільні фікстури (tests/conftest.py).

Нижче наведено вихідний код ключової доменної моделі Student (src/student_manager/models.py), що реалізує сувору валідацію обов'язкових полів та меж оцінок:

```python
@dataclass
class Student:
    first_name: str
    last_name: str
    group: str
    average_grade: float
    student_id: int | None = None

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError("Name, surname and group cannot be empty")
        if not 0 <= self.average_grade <= 100:
            raise ValueError("Average grade must be between 0 and 100")
        if self.student_id is not None:
            # перевіряємо шоб айді був цілим додатнім числом і не bool
            if isinstance(self.student_id, bool) or not isinstance(
                self.student_id, int
            ):
                raise TypeError("Student ID must be an integer")
            if self.student_id <= 0:
                raise ValueError("Student ID must be a positive integer")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

Клас StudentRegistry (src/student_manager/registry.py) інкапсулює роботу з репозиторієм та відправку нотифікацій. Наводжу реалізацію методу додавання студента add:

```python
def add(self, student_id: int, student: Student) -> Student:
    # валідуєм id і додаєм копію студента з узгодженим id
    if (
        isinstance(student_id, bool)
        or not isinstance(student_id, int)
        or student_id <= 0
    ):
        raise ValueError("student_id must be a positive integer")

    existing_id = student.student_id
    if existing_id is not None and existing_id != student_id:
        raise ValueError("Inconsistent student_id between entity and registry key")

    stored_student = replace(student, student_id=student_id)

    self._repository.add(student_id, stored_student)
    self._notifier.send(f"Added student {student_id}: {stored_student.full_name}")
    return stored_student
```

Для демонстрації структури вихідних файлів та виявлення всіх тестових функцій фреймворком pytest я виконав команду виявлення тестів pytest --collect-only:

```
PS F:\university\3yr-1\professional_python> pytest --collect-only -q --ignore=tests/test_persistence.py
tests/integration/test_student_pipeline.py::test_full_pipeline_csv_to_json_export_with_config
tests/integration/test_student_pipeline.py::test_pipeline_tolerant_mode_skips_malformed_csv_rows
tests/integration/test_student_pipeline.py::test_pipeline_strict_mode_aborts_on_malformed_csv
tests/integration/test_student_pipeline.py::test_end_to_end_registry_workflow_with_export
tests/test_analytics.py::AnalyticsTests::test_functional_extensions
tests/test_analytics.py::AnalyticsTests::test_summary_keeps_duplicate_surnames_and_strict_threshold
tests/test_analytics.py::AnalyticsTests::test_timing_decorator_preserves_result_and_metadata
tests/test_application_coverage.py::DemosCoverageTests::test_run_lab2_handles_empty_student_list
tests/test_application_coverage.py::DemosCoverageTests::test_run_lab2_outputs_all_analytics
tests/test_application_coverage.py::Lab4DemoCoverageTests::test_lab4_demo_main_block
tests/test_application_coverage.py::Lab4DemoCoverageTests::test_run_demo_executes_successfully
tests/test_application_coverage.py::MainApplicationCoverageTests::test_create_demo_students
[Проміжні рядки пропущено; повний протокол збережено локально]
tests/unit/test_student_behaviors.py::test_student_id_type_and_positivity_validation[False-TypeError]
tests/unit/test_student_behaviors.py::test_student_id_type_and_positivity_validation[1.5-TypeError]
tests/unit/test_student_behaviors.py::test_grade_finite_boundaries_and_rejections
tests/unit/test_student_behaviors.py::test_academic_record_empty_grades_returns_none
tests/unit/test_student_behaviors.py::test_academic_record_average_calculation_with_pytest_approx
tests/unit/test_student_behaviors.py::test_academic_record_add_grade_immutability
tests/unit/test_student_behaviors.py::test_academic_record_rejects_invalid_student_id
tests/unit/test_student_behaviors.py::test_parse_student_malformed_csv_row_raises_record_validation_error
tests/unit/test_student_behaviors.py::test_read_csv_rows_header_schema_mismatch_raises_data_import_error
tests/unit/test_student_behaviors.py::test_read_csv_rows_missing_file_raises_data_import_error
tests/unit/test_student_behaviors.py::test_export_students_json_creates_valid_file
tests/unit/test_student_behaviors.py::test_load_config_valid_yaml_creates_app_config
tests/unit/test_student_behaviors.py::test_load_config_malformed_yaml_raises_configuration_error
tests/unit/test_student_behaviors.py::test_load_config_colliding_paths_raises_configuration_error
tests/unit/test_student_behaviors.py::test_config_with_monkeypatch_cwd
tests/unit/test_student_behaviors.py::test_monkeypatch_function_behavior
```

Продовження того самого виводу:

```
224 tests collected in 0.82s
```

2. Мінімальна матриця тестування поведінок (Test Matrix)

Згідно з розділом 91 лабораторного завдання я розробив систематичну матрицю тестування, яка охоплює всі обов'язкові категорії поведінки системи: нормальні випадки (Normal), граничні мінімуми і максимуми (Boundary), порожні колекції та рядки (Empty), невалідні типи і значення (Invalid), відсутні файли або секції (Missing data), а також збої зовнішніх залежностей (Dependency failure). Зведена матриця представлена в таблиці:

Компонент / Метод	Категорія тесту	Тестовий сценарій / Вхідні дані	Очікувана поведінка / Assertion	Тестова функція
Student (dataclass)	Normal	Іван Мельник, ФеП-33, grade=88.5	Екземпляр валідний, full_name='Іван Мельник'	test_student_valid_construction_and_full_name
Student (dataclass)	Boundary min	average_grade = 0.0	Успішне створення, grade == 0.0	test_student_grade_boundaries_and_invalid_values
Student (dataclass)	Boundary max	average_grade = 100.0	Успішне створення, grade == 100.0	test_student_grade_boundaries_and_invalid_values

Student (dataclass)	Empty field	first_name='', last_name=' '	pytest.raises(ValueError, match='non-empty')	test_student_required_fields_reject_empty
Student (dataclass)	Invalid value	average_grade = 105.0, -5.0, NaN	pytest.raises(ValueError, match='[0..100]')	test_student_grade_boundaries_and_invalid_values
AcademicRecord	Empty	grades = ()	average() повертає None	test_academic_record_empty_grades_returns_none
AcademicRecord	Approx calculation	grades = (85.0, 92.5, 78.0)	average == pytest.approx(85.16666, abs=1e-3)	test_academic_record_average_calculation_with_pytest_approx
StudentRegistry	Dependency Mock	Mock repository, return_value	repo.add викликано 1 раз із валідним ID	test_registry_repository_mock_return_value
StudentRegistry	Notifier Mock	Mock notifier, interaction	notifier.send.assert_called_once_with(...)	test_registry_add_notifier_mock_interactions
StudentRegistry	Dependency failure	repo.add.side_effect = DatabaseError	Виняток прокидається, notifier НЕ викликається	test_registry_add_dependency_side_effect_failure...
ReliableIO (CSV)	Malformed CSV	Пропущене поле або некоректний бал	pytest.raises(RecordValidationError)	test_parse_student_malformed_csv_row_raises...
ReliableIO (JSON)	tmp_path file	Експорт 4 студентів у JSON	Файл створено в ізольованій tmp_path теці	test_export_students_json_creates_valid_file
AppConfig (YAML)	Missing data / file	Файл config.yaml відсутній	pytest.raises(ConfigurationError, match='not found')	test_load_config_malformed_yaml_raises...
AppConfig (YAML)	Monkeypatch env/cwd	monkeypatch.chdir(tmp_path)	Відносні шляхи резолвляться відносно ізоляції	test_config_with_monkeypatch_cwd
AsyncStudentService	AsyncMock await	AsyncMock repository та notifier	await repo.get(1); await notifier.notify(...)	test_load_success_returns_student_and_notifies

3. Валідація моделі, граничні значення, параметризація, pytest.raises та pytest.approx

У модулі tests/unit/test_student_behaviors.py я реалізував перевірку коректного створення сутності Student, використовуючи параметризацію pytest.mark.parametrize для граничних значень середнього бала (0.0 та 100.0), а також перехоплення винятків pytest.raises для неприпустимих чисел (105.0, -5.0,

нескінченність, NaN). Крім того, для тестування обчислення середнього бала в класі AcademicRecord використано pytest.approx для безпечного порівняння чисел із рухомою комою:

```python
@pytest.mark.unit
def test_student_valid_construction_and_full_name() -> None:
    # перевірка створення студента з валідними даними
    student = Student(
        first_name="Остап",
        last_name="Бендер",
        group="ФеП-33",
        average_grade=89.5,
        student_id=10,
    )
    assert student.first_name == "Остап"
    assert student.last_name == "Бендер"
    assert student.group == "ФеП-33"
    assert student.average_grade == pytest.approx(89.5)
    assert student.student_id == 10
    assert student.full_name == "Остап Бендер"
```

```python
@pytest.mark.unit
@pytest.mark.parametrize(
    "grade,is_valid",
    [
        (0.0, True),
        (100.0, True),
        (50.5, True),
        (-0.1, False),
        (100.1, False),
        (-25.0, False),
        (150.0, False),
        (float("nan"), False),
        (float("inf"), False),
        (float("-inf"), False),
    ],
)
def test_student_grade_boundaries_and_invalid_values(
    grade: float, is_valid: bool
) -> None:
    # перевірка меж оцінки від 0 до 100 та відхилення нескінченних і nan значень
    if is_valid:
        st = Student("Тарас", "Шевченко", "ФеП-31", grade, student_id=2)
        assert st.average_grade == pytest.approx(grade)
    else:
        with pytest.raises(ValueError, match="Average grade must be between 0 and 100"):
            Student("Тарас", "Шевченко", "ФеП-31", grade, student_id=2)
```

```python
@pytest.mark.unit
def test_academic_record_average_calculation_with_pytest_approx() -> None:
    # розрахунок середнього бала через pytest.approx для дійсних чисел
    grades = (Grade(85.5), Grade(92.3), Grade(78.1))
    rec = AcademicRecord(student_id=5, grades=grades)
    expected_avg = (85.5 + 92.3 + 78.1) / 3
    assert rec.average is not None
    assert rec.average == pytest.approx(expected_avg, abs=1e-6)
    assert len(rec) == 3
```

```python
@pytest.mark.unit
def test_academic_record_empty_grades_returns_none() -> None:
    # порожня картка студента повертає None для середнього бала
    rec = AcademicRecord(student_id=5)
    assert rec.average is None
    assert len(rec) == 0
```

Результати виконання зазначених модульних перевірок валідації моделі та розрахунків:

```
PS F:\university\3yr-1\professional_python> pytest tests/unit/test_student_behaviors.py -k student or
academic -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 37 items

tests/unit/test_student_behaviors.py::test_student_valid_construction_and_full_name PASSED [  2%]
tests/unit/test_student_behaviors.py::test_student_required_fields_reject_empty[-
\u0411\u0435\u043d\u0434\u0435\u0440-\u0424\u0435\u041f-33] PASSED [  5%]
tests/unit/test_student_behaviors.py::test_student_required_fields_reject_empty[  -
\u0411\u0435\u043d\u0434\u0435\u0440-\u0424\u0435\u041f-33] PASSED [  8%]
tests/unit/test_student_behaviors.py::test_student_required_fields_reject_empty[\u041e\u0441\u0442\u0430\u0
43f--\u0424\u0435\u041f-33] PASSED [ 10%]
[Проміжні рядки пропущено; повний протокол збережено локально]
tests/unit/test_student_behaviors.py::test_student_id_type_and_positivity_validation[False-TypeError]
PASSED [ 59%]
tests/unit/test_student_behaviors.py::test_student_id_type_and_positivity_validation[1.5-TypeError] PASSED
[ 62%]
tests/unit/test_student_behaviors.py::test_grade_finite_boundaries_and_rejections PASSED [ 64%]
tests/unit/test_student_behaviors.py::test_academic_record_empty_grades_returns_none PASSED [ 67%]
tests/unit/test_student_behaviors.py::test_academic_record_average_calculation_with_pytest_approx PASSED [
70%]
tests/unit/test_student_behaviors.py::test_academic_record_add_grade_immutability PASSED [ 72%]
tests/unit/test_student_behaviors.py::test_academic_record_rejects_invalid_student_id PASSED [ 75%]
tests/unit/test_student_behaviors.py::test_parse_student_malformed_csv_row_raises_record_validation_error
PASSED [ 78%]
tests/unit/test_student_behaviors.py::test_read_csv_rows_header_schema_mismatch_raises_data_import_error
PASSED [ 81%]
tests/unit/test_student_behaviors.py::test_read_csv_rows_missing_file_raises_data_import_error PASSED [
83%]
tests/unit/test_student_behaviors.py::test_export_students_json_creates_valid_file PASSED [ 86%]
tests/unit/test_student_behaviors.py::test_load_config_valid_yaml_creates_app_config PASSED [ 89%]
tests/unit/test_student_behaviors.py::test_load_config_malformed_yaml_raises_configuration_error PASSED [
91%]
tests/unit/test_student_behaviors.py::test_load_config_colliding_paths_raises_configuration_error PASSED [
94%]
tests/unit/test_student_behaviors.py::test_config_with_monkeypatch_cwd PASSED [ 97%]
tests/unit/test_student_behaviors.py::test_monkeypatch_function_behavior PASSED [100%]
```

Продовження того самого виводу:

```
============================== 37 passed in 0.90s ==============================
```

4. StudentRegistry, репозиторій та Notifier через Mock: return_value, side_effect та перевірка взаємодій

Для тестування класу StudentRegistry в ізоляції від зовнішніх компонентів у tests/unit/test_registry_interactions.py я використав об'єкти Mock із бібліотеки unittest.mock. Було налаштовано повернені значення (return_value) для перевірки успішного додавання, верифіковано взаємодії через assert_called_once_with, а також протестовано failure path зовнішньої залежності через side_effect, де перевірено, що в разі помилки репозиторію сповіщувач Notifier гарантовано не викликається:

```python
@pytest.mark.unit
def test_registry_repository_mock_return_value(sample_student: Student) -> None:
    # використання Mock із налаштованим return_value для ізоляції репозиторію
    mock_repo = Mock(spec=Repository)
    mock_repo.get.return_value = sample_student
    mock_notifier = Mock(spec=Notifier)

    registry = StudentRegistry(repository=mock_repo, notifier=mock_notifier)
    student = registry.get_by_id(1)

    assert student == sample_student
    mock_repo.get.assert_called_once_with(1)

    mock_repo.all.return_value = (sample_student,)
    all_students = list(iter(registry))
    assert all_students == [sample_student]
    mock_repo.all.assert_called_once()
```

```python
@pytest.mark.unit
def test_registry_add_notifier_mock_interactions(sample_student: Student) -> None:
    # перевірка взаємодії з нотифікатором через Mock interaction assertions
    mock_repo = Mock(spec=Repository)
    mock_notifier = Mock(spec=Notifier)

    registry = StudentRegistry(repository=mock_repo, notifier=mock_notifier)
    added = registry.add(1, sample_student)

    assert added.student_id == 1
    mock_repo.add.assert_called_once_with(1, added)
    mock_notifier.send.assert_called_once_with("Added student 1: Остап Бендер")
```

```python
@pytest.mark.unit
def test_registry_add_dependency_side_effect_failure_assert_notifier_not_called(
    sample_student: Student,
) -> None:
    # збій зовнішньої залежності через side_effect блокує виклик нотифікатора
    mock_repo = Mock(spec=Repository)
    mock_repo.add.side_effect = RuntimeError("Storage connection failed")
    mock_notifier = Mock(spec=Notifier)

    registry = StudentRegistry(repository=mock_repo, notifier=mock_notifier)

    with pytest.raises(RuntimeError, match="Storage connection failed"):
        registry.add(1, sample_student)

    mock_notifier.send.assert_not_called()
```

Виконання тестів взаємодії з мок-об'єктами реєстру:

```
PS F:\university\3yr-1\professional_python> pytest tests/unit/test_registry_interactions.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 11 items

tests/unit/test_registry_interactions.py::test_search_by_last_name_trims_and_ignores_case PASSED [  9%]
tests/unit/test_registry_interactions.py::test_search_by_id_and_group PASSED [ 18%]
tests/unit/test_registry_interactions.py::test_find_best_student_and_average_empty_registry PASSED [ 27%]
tests/unit/test_registry_interactions.py::test_find_best_student_and_average_with_students PASSED [ 36%]
tests/unit/test_registry_interactions.py::test_registry_repository_mock_return_value PASSED [ 45%]
tests/unit/test_registry_interactions.py::test_registry_add_notifier_mock_interactions PASSED [ 54%]
tests/unit/test_registry_interactions.py::test_registry_edit_notifier_mock_interactions PASSED [ 63%]
tests/unit/test_registry_interactions.py::test_registry_add_dependency_side_effect_failure_assert_notifier_
not_called PASSED [ 72%]
tests/unit/test_registry_interactions.py::test_registry_edit_dependency_side_effect_failure_assert_notifier
_not_called PASSED [ 81%]
tests/unit/test_registry_interactions.py::test_reporting_service_summary_and_export PASSED [ 90%]
tests/unit/test_registry_interactions.py::test_registry_contains_and_getitem_dunders PASSED [100%]

============================== 11 passed in 0.26s ==============================
```

5. Файлові тести через tmp_path, валідація CSV, кастомні винятки та JSON-експорт

Для забезпечення повної ізоляції файлових операцій від локальної системи я застосував стандартну фікстуру tmp_path. У тестах перевірено роботу з некоректними даними CSV (збудження доменного винятку RecordValidationError), експорт списку студентів у валідний JSON-файл, а також поведінку конвеєра в режимах strict (негайне переривання) та tolerant (пропуск невалідних рядків із фіксацією у журналі):

```python
@pytest.mark.unit
def test_parse_student_malformed_csv_row_raises_record_validation_error() -> None:
    # кастомний виняток RecordValidationError для некоректних рядків csv
    bad_id_row = {
        "student_id": "-5",
        "first_name": "Іван",
        "last_name": "Франко",
        "group": "ФеП-32",
        "average_grade": "85.0",
    }
    with pytest.raises(RecordValidationError) as exc_info:
        parse_student(bad_id_row, line_number=3)
    assert exc_info.value.line_number == 3
    assert exc_info.value.field == "student_id"

    bad_width_row = {"_width_error": "expected 5 columns, got 3"}
    with pytest.raises(RecordValidationError) as exc_width:
        parse_student(bad_width_row, line_number=7)
    assert exc_width.value.line_number == 7
    assert exc_width.value.field == "row"

    nan_grade_row = {
        "student_id": "1",
        "first_name": "Іван",
        "last_name": "Франко",
        "group": "ФеП-32",
        "average_grade": "nan",
    }
    with pytest.raises(RecordValidationError) as exc_nan:
        parse_student(nan_grade_row, line_number=9)
    assert exc_nan.value.line_number == 9
    assert exc_nan.value.field == "average_grade"
```

```python
@pytest.mark.unit
def test_export_students_json_creates_valid_file(
    tmp_path: Path, sample_students: list[Student]
) -> None:
    # атомарний експорт списку студентів у json
    out_file = tmp_path / "exported.json"
    exported_count = export_students_json(sample_students, out_file)
    assert exported_count == len(sample_students)
    assert out_file.exists()

    loaded = json.loads(out_file.read_text(encoding="utf-8"))
    assert len(loaded) == 4
    assert loaded[0]["first_name"] == "Остап"
    assert loaded[0]["average_grade"] == pytest.approx(88.5)
```

```python
@pytest.mark.integration
def test_pipeline_tolerant_mode_skips_malformed_csv_rows(tmp_path: Path) -> None:
    # толерантний режим: пропускає биті рядки, логує помилки та зберігає валідні
    input_csv = tmp_path / "mixed_students.csv"
    input_csv.write_text(
        "student_id,first_name,last_name,group,average_grade\n"
        "1,Остап,Бендер,ФеП-33,90.0\n"
        "2,Пошкоджений,Рядок\n"
        "3,Тарас,Шевченко,ФеП-31,95.0\n"
        "4,Некоректна,Оцінка,ФеП-33,не_число\n"
        "-5,Від'ємний,Айді,ФеП-32,80.0\n"
        "6,Іван,Франко,ФеП-32,70.0\n",
        encoding="utf-8",
    )
    output_json = tmp_path / "tolerant_out.json"
    log_file = tmp_path / "tolerant.log"
    config_file = tmp_path / "tolerant_config.yaml"
    config_file.write_text(
        f"""
input:
  path: "{input_csv.name}"
output:
  path: "{output_json.name}"
processing:
  minimum_grade: 80.0
  skip_invalid: true
logging:
  level: "WARNING"
  path: "{log_file.name}"
""",
        encoding="utf-8",
    )

    app_config = load_config(config_file)
    logger = configure_logging(app_config.logging, logger_name="test_pipeline_tolerant")
```

```python
    try:
        stats = process_config(app_config, logger=logger)
    finally:
        close_logging_handlers(logger)

    assert stats.total == 6
    assert stats.valid == 3
    assert stats.invalid == 3
    assert stats.exported == 2

    exported_data = json.loads(output_json.read_text(encoding="utf-8"))
    assert len(exported_data) == 2
    ids = {item["student_id"] for item in exported_data}
    assert ids == {1, 3}

    assert log_file.exists()
    logs = log_file.read_text(encoding="utf-8")
    assert "Invalid record at line" in logs
```

```python
@pytest.mark.integration
def test_pipeline_strict_mode_aborts_on_malformed_csv(tmp_path: Path) -> None:
    # суворий режим: перериває роботу на першій помилці та не створює битих файлів
    input_csv = tmp_path / "corrupted.csv"
    input_csv.write_text(
        "student_id,first_name,last_name,group,average_grade\n"
        "1,Остап,Бендер,ФеП-33,90.0\n"
        "2,Тарас,Шевченко,ФеП-31,invalid_grade\n",
        encoding="utf-8",
    )
    output_json = tmp_path / "strict_out.json"
    log_file = tmp_path / "strict.log"
    config_file = tmp_path / "strict_config.yaml"
    config_file.write_text(
        f"""
input:
  path: "{input_csv.name}"
output:
  path: "{output_json.name}"
processing:
  minimum_grade: 60.0
  skip_invalid: false
logging:
  level: "INFO"
  path: "{log_file.name}"
""",
        encoding="utf-8",
    )

    app_config = load_config(config_file)
    logger = configure_logging(app_config.logging, logger_name="test_pipeline_strict")
```

```python
    try:
        with pytest.raises(RecordValidationError) as exc_info:
            process_config(app_config, logger=logger)
    finally:
        close_logging_handlers(logger)

    assert exc_info.value.line_number == 3
    assert exc_info.value.field == "average_grade"
    assert not output_json.exists()
```

Результати виконання файлових та інтеграційних тестів конвеєра імпорту/експорту:

```
PS F:\university\3yr-1\professional_python> pytest tests/integration/test_student_pipeline.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 4 items

tests/integration/test_student_pipeline.py::test_full_pipeline_csv_to_json_export_with_config PASSED [ 25%]
tests/integration/test_student_pipeline.py::test_pipeline_tolerant_mode_skips_malformed_csv_rows PASSED [
50%]
tests/integration/test_student_pipeline.py::test_pipeline_strict_mode_aborts_on_malformed_csv PASSED [ 75%]
tests/integration/test_student_pipeline.py::test_end_to_end_registry_workflow_with_export PASSED [100%]

============================== 4 passed in 0.32s ==============================
```

6. Конфігурація застосунку YAML та monkeypatch

Управління параметрами конвеєра реалізовано через YAML-конфігурацію (AppConfig). Для тестування поведінки при зміні робочої директорії та підміни внутрішніх функцій я використав фікстуру monkeypatch. Це дозволяє

детерміновано перевіряти fail-fast валідацію відносних шляхів та обробку синтаксично пошкодженого YAML без впливу на системне оточення:

```python
@pytest.mark.unit
def test_load_config_valid_yaml_creates_app_config(tmp_path: Path) -> None:
    # завантаження валідної конфігурації yaml
    input_file = tmp_path / "students.csv"
    input_file.write_text(
        "student_id,first_name,last_name,group,average_grade\n1,Остап,Бендер,ФеП-33,90.0\n",
        encoding="utf-8",
    )
    output_file = tmp_path / "output.json"
    log_file = tmp_path / "app.log"
    config_file = tmp_path / "config.yaml"
    config_file.write_text(
        f"""
input:
  path: "{input_file.name}"
output:
  path: "{output_file.name}"
processing:
  minimum_grade: 85.0
  skip_invalid: true
logging:
  level: "INFO"
  path: "{log_file.name}"
""",
        encoding="utf-8",
    )

    cfg = load_config(config_file)
    assert cfg.processing.minimum_grade == pytest.approx(85.0)
    assert cfg.processing.skip_invalid is True
    assert cfg.logging.level == "INFO"
    assert cfg.input_path == input_file.resolve()
```

```python
@pytest.mark.unit
def test_config_with_monkeypatch_cwd(
    tmp_path: Path, monkeypatch: pytest.MonkeyPatch
) -> None:
    # перевірка роботи з поточною робочою директорією через monkeypatch
    input_file = tmp_path / "in.csv"
    input_file.write_text(
        "student_id,first_name,last_name,group,average_grade\n1,Остап,Бендер,ФеП-33,90.0\n",
        encoding="utf-8",
    )
    config_file = tmp_path / "config.yaml"
    config_file.write_text(
        """
input:
  path: "in.csv"
output:
  path: "out.json"
processing:
  minimum_grade: 75.0
  skip_invalid: false
logging:
  level: "DEBUG"
  path: "test.log"
""",
        encoding="utf-8",
    )

    monkeypatch.chdir(tmp_path)
    cfg = load_config(Path("config.yaml"))
    assert cfg.input_path == input_file.resolve()
    assert cfg.processing.minimum_grade == pytest.approx(75.0)
```

```python
@pytest.mark.unit
def test_monkeypatch_function_behavior(
    tmp_path: Path, monkeypatch: pytest.MonkeyPatch, sample_student: Student
) -> None:
    # підміна функції student_to_dict через monkeypatch
    def custom_to_dict(st: Student) -> dict[str, Any]:
        return {
            "custom_id": st.student_id,
            "display": f"{st.last_name.upper()}, {st.first_name}",
            "passed": st.average_grade >= 60.0,
        }

    monkeypatch.setattr("student_manager.reliable_io.student_to_dict", custom_to_dict)

    out_file = tmp_path / "custom_out.json"
    export_students_json([sample_student], out_file)

    payload = json.loads(out_file.read_text(encoding="utf-8"))
    assert len(payload) == 1
    assert payload[0]["display"] == "БЕНДЕР, Остап"
    assert payload[0]["passed"] is True
    assert "custom_id" in payload[0]
```

Результати перевірки модуля конфігурації із застосуванням monkeypatch:

```
PS F:\university\3yr-1\professional_python> pytest tests/unit/test_student_behaviors.py -k config or
monkeypatch -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 37 items / 32 deselected / 5 selected

tests/unit/test_student_behaviors.py::test_load_config_valid_yaml_creates_app_config PASSED [ 20%]
tests/unit/test_student_behaviors.py::test_load_config_malformed_yaml_raises_configuration_error PASSED [
40%]
tests/unit/test_student_behaviors.py::test_load_config_colliding_paths_raises_configuration_error PASSED [
60%]
tests/unit/test_student_behaviors.py::test_config_with_monkeypatch_cwd PASSED [ 80%]
tests/unit/test_student_behaviors.py::test_monkeypatch_function_behavior PASSED [100%]

====================== 5 passed, 32 deselected in 0.48s =======================
```

7. Асинхронний сервіс та AsyncMock для зовнішніх залежностей

Відповідно до вимог індивідуального варіанта №1 у проєкті реалізовано повноцінний асинхронний сервіс AsyncStudentService (src/student_manager/async_service.py), який очікує асинхронний репозиторій і викликає синхронний Notifier. Для його тестування в tests/test_async_service.py використано AsyncMock. Перевірено коректне очікування корутин через await, підтверджено відсутність виклику нотифікатора при помилках репозиторію, а також протестовано реальну in-memory асинхронну взаємодію:

```python
async def load(self, student_id: int) -> Student | None:
    # валідуєм ідентифікатор студента
    if (
        isinstance(student_id, bool)
        or not isinstance(student_id, int)
        or student_id <= 0
    ):
        raise ValueError("student_id must be a positive integer")

    student = await self._repository.get(student_id)
    if student is not None:
        self._notifier.send(f"Loaded student {student_id}: {student.full_name}")
    return student
```

```python
def test_load_success_returns_student_and_notifies(self) -> None:
    # успішне завантаження з сповіщенням
    self.repository.get.return_value = self.sample_student

    result = asyncio.run(self.service.load(1))

    self.assertEqual(result, self.sample_student)
    self.repository.get.assert_awaited_once_with(1)
    self.notifier.send.assert_called_once_with("Loaded student 1: Олена Коваль")
```

```python
def test_load_repository_failure_propagates_and_does_not_notify(self) -> None:
    # помилка репозиторію прокидається наверх
    self.repository.get.side_effect = RuntimeError("database connection failed")

    with self.assertRaises(RuntimeError) as ctx:
        asyncio.run(self.service.load(1))

    self.assertEqual(str(ctx.exception), "database connection failed")
    self.repository.get.assert_awaited_once_with(1)
    self.notifier.send.assert_not_called()
```

```python
def test_real_async_repository_flow(self) -> None:
    # повний сценарій із реальною асинхронною імплементацією
    repo = InMemoryAsyncStudentRepository({1: self.sample_student})
    notifier = RecordingNotifier()
    service = AsyncStudentService(repo, notifier)

    loaded = asyncio.run(service.load(1))
    self.assertEqual(loaded, self.sample_student)
    self.assertEqual(notifier.messages, ["Loaded student 1: Олена Коваль"])

    missing = asyncio.run(service.load(99))
    self.assertIsNone(missing)
    self.assertEqual(notifier.messages, ["Loaded student 1: Олена Коваль"])
```

Результати виконання асинхронного тестового набору:

```
PS F:\university\3yr-1\professional_python> pytest tests/test_async_service.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 6 items

tests/test_async_service.py::AsyncStudentServiceTests::test_load_invalid_ids_raise_value_error_without_awai
ting
tests/test_async_service.py::AsyncStudentServiceTests::test_load_invalid_ids_raise_value_error_without_awai
ting PASSED [ 16%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_missing_student_returns_none_and_does_not_
notify PASSED [ 33%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_notifier_failure_propagates PASSED [ 50%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_repository_failure_propagates_and_does_not
_notify PASSED [ 66%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_success_returns_student_and_notifies
PASSED [ 83%]
tests/test_async_service.py::AsyncStudentServiceTests::test_real_async_repository_flow PASSED [100%]

==================== 6 passed, 9 subtests passed in 0.26s =====================
```

8. Життєвий цикл фікстур із yield, очищення (teardown) та аналіз рівнів ізоляції (scopes)

У файлі tests/conftest.py визначено рівно 7 модульних фікстур. Усі вони свідомо використовують стандартний рівень ізоляції scope='function'. Це забезпечує сувору детермінованість: кожен окремий тест отримує новий незалежний екземпляр об'єкта, що повністю виключає міжтестові витоки стану. Для ресурсів, що потребують фіналізації (очищення черги повідомлень нотифікатора, скидання in-memory сховища репозиторію, відновлення початкової поточної директорії), застосовано оператор yield, після якого виконуються обов'язкові інструкції teardown:

```python
@pytest.fixture
def recording_notifier() -> Iterator[RecordingNotifier]:
    # фікстура нотифікатора із очищенням повідомлень на teardown
    notifier = RecordingNotifier()
    yield notifier
    notifier.messages.clear()
```

```python
@pytest.fixture
def in_memory_repository() -> Iterator[InMemoryRepository[Student]]:
    # фікстура репозиторію з очищенням сховища на teardown
    repo: InMemoryRepository[Student] = InMemoryRepository()
    yield repo
    repo._items.clear()
```

```python
@pytest.fixture
def populated_registry(
    sample_students: list[Student],
    recording_notifier: RecordingNotifier,
) -> Iterator[tuple[StudentRegistry, InMemoryRepository[Student], RecordingNotifier]]:
    # наповнений реєстр із перевіркою та teardown очищенням
    repo: InMemoryRepository[Student] = InMemoryRepository()
    registry = StudentRegistry(repository=repo, notifier=recording_notifier)
    for st in sample_students:
        assert st.student_id is not None
        registry.add(st.student_id, st)
    yield registry, repo, recording_notifier
    repo._items.clear()
    recording_notifier.messages.clear()
```

```python
@pytest.fixture
def isolated_cwd(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> Iterator[Path]:
    # ізоляція робочої директорії через monkeypatch з teardown відновленням
    monkeypatch.chdir(tmp_path)
    yield tmp_path
```

Теоретично pytest підтримує 5 рівнів fixture scopes: function (дефолтний, ізоляція кожного тесту), class (один раз на клас), module (один раз на тестовий файл), package (один раз на пакет) та session (один раз на весь тестовий прогін). У даному проєкті використання function scope для всіх 7 фікстур є найбільш надійним інженерним вибором, оскільки запобігає будь-яким каскадним сайд-ефектам.

9. Результати виконання тестового набору (passed/failed)

Усі числові метрики в цьому звіті споживаються виключно з артефакту final_results.json. Загалом у тестовому наборі успішно виконано 224 тестів, кількість невдалих тестів (failed) становить 0. Нижче наведено протоколи виконання як сфокусованого набору модульних/інтеграційних тестів, так і повного зведеного запуску:

1) Сфокусований запуск цільових тестів варіанта (tests/unit, tests/integration, test_async_service.py):

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest tests/unit tests/integration
tests/test_async_service.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 58 items

tests/unit/test_registry_interactions.py::test_search_by_last_name_trims_and_ignores_case PASSED [  1%]
tests/unit/test_registry_interactions.py::test_search_by_id_and_group PASSED [  3%]
tests/unit/test_registry_interactions.py::test_find_best_student_and_average_empty_registry PASSED [  5%]
tests/unit/test_registry_interactions.py::test_find_best_student_and_average_with_students PASSED [  6%]
[Проміжні рядки пропущено; повний протокол збережено локально]
tests/unit/test_student_behaviors.py::test_load_config_valid_yaml_creates_app_config PASSED [ 75%]
tests/unit/test_student_behaviors.py::test_load_config_malformed_yaml_raises_configuration_error PASSED [
77%]
tests/unit/test_student_behaviors.py::test_load_config_colliding_paths_raises_configuration_error PASSED [
79%]
tests/unit/test_student_behaviors.py::test_config_with_monkeypatch_cwd PASSED [ 81%]
tests/unit/test_student_behaviors.py::test_monkeypatch_function_behavior PASSED [ 82%]
tests/integration/test_student_pipeline.py::test_full_pipeline_csv_to_json_export_with_config PASSED [ 84%]
tests/integration/test_student_pipeline.py::test_pipeline_tolerant_mode_skips_malformed_csv_rows PASSED [
86%]
tests/integration/test_student_pipeline.py::test_pipeline_strict_mode_aborts_on_malformed_csv PASSED [ 87%]
tests/integration/test_student_pipeline.py::test_end_to_end_registry_workflow_with_export PASSED [ 89%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_invalid_ids_raise_value_error_without_awai
ting
tests/test_async_service.py::AsyncStudentServiceTests::test_load_invalid_ids_raise_value_error_without_awai
ting PASSED [ 91%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_missing_student_returns_none_and_does_not_
notify PASSED [ 93%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_notifier_failure_propagates PASSED [ 94%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_repository_failure_propagates_and_does_not
_notify PASSED [ 96%]
tests/test_async_service.py::AsyncStudentServiceTests::test_load_success_returns_student_and_notifies
PASSED [ 98%]
tests/test_async_service.py::AsyncStudentServiceTests::test_real_async_repository_flow PASSED [100%]
```

Продовження того самого виводу:

```
==================== 58 passed, 9 subtests passed in 1.88s ====================
```

2) Повний запуск тестового набору всього проєкту з вимірюванням покриття:

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest -q --cov=student_manager --cov-branch --cov-
report=term-missing --cov-report=json:.local_workflow/lab6/final_coverage.json --cov-
report=html:.local_workflow/lab6/htmlcov
..................................................................................... [ 43%]
..................................................................... [ 74%]
.....................................................              [100%]
============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.14-final-0 _____________

Name                                          Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------------
src\student_manager\__init__.py                   1      0      0      0   100%
src\student_manager\analytics.py                 33      0      6      0   100%
src\student_manager\async_service.py             16      0      4      0   100%
src\student_manager\config.py                   107     15     50     14    82%   37, 40, 45, 53, 61, 68,
88, 90, 103, 128, 135, 143, 145, 163, 165
src\student_manager\decorators.py                16      0      0      0   100%
src\student_manager\demos.py                     19      0      2      0   100%
src\student_manager\design_experiments.py        71      6      8      4    87%   42, 55, 57, 60-61, 63
src\student_manager\domain.py                    74      4     18      2    93%   36-37, 69, 81
src\student_manager\exceptions.py                11      0      0      0   100%
src\student_manager\experiments.py               34      5     10      1    86%   47-50, 54
src\student_manager\lab3_demo.py                 93      7     22      4    90%   28-29, 64->67, 69->75,
143-146, 150
src\student_manager\lab4_demo.py                 81      0      8      0   100%
src\student_manager\lab5_demo.py                 27      3      2      1    86%   50-51, 55
src\student_manager\main.py                      87      0     36      0   100%
src\student_manager\models.py                    24      0     10      0   100%
src\student_manager\registry.py                  96      3     28      3    95%   148, 153, 171
src\student_manager\reliable_experiments.py     199      6     38      5    95%   150-151, 157->161, 298,
331-332, 333->338, 450
src\student_manager\reliable_io.py              201      8     44      4    95%   53, 103, 144-145, 152,
224-225, 249, 253->262, 388->391
src\student_manager\repositories.py              34      0      4      0   100%
src\student_manager\services.py                  15      0      2      0   100%
src\student_manager\streaming.py                159      4     66      3    97%   48-49, 70->72, 73, 104
src\student_manager\streaming_experiments.py    135     16     22      6    85%   187->193, 235, 266->240,
270-275, 280->284, 288-317, 326
```

Продовження того самого виводу:

```
-----------------------------------------------------------------------------------------
TOTAL                                          1533     77    380     47    93%
Coverage HTML written to dir .local_workflow/lab6/htmlcov
Coverage JSON written to file .local_workflow/lab6/final_coverage.json
Required test coverage of 80.0% reached. Total coverage: 93.41%
224 passed, 50 subtests passed in 26.39s
```

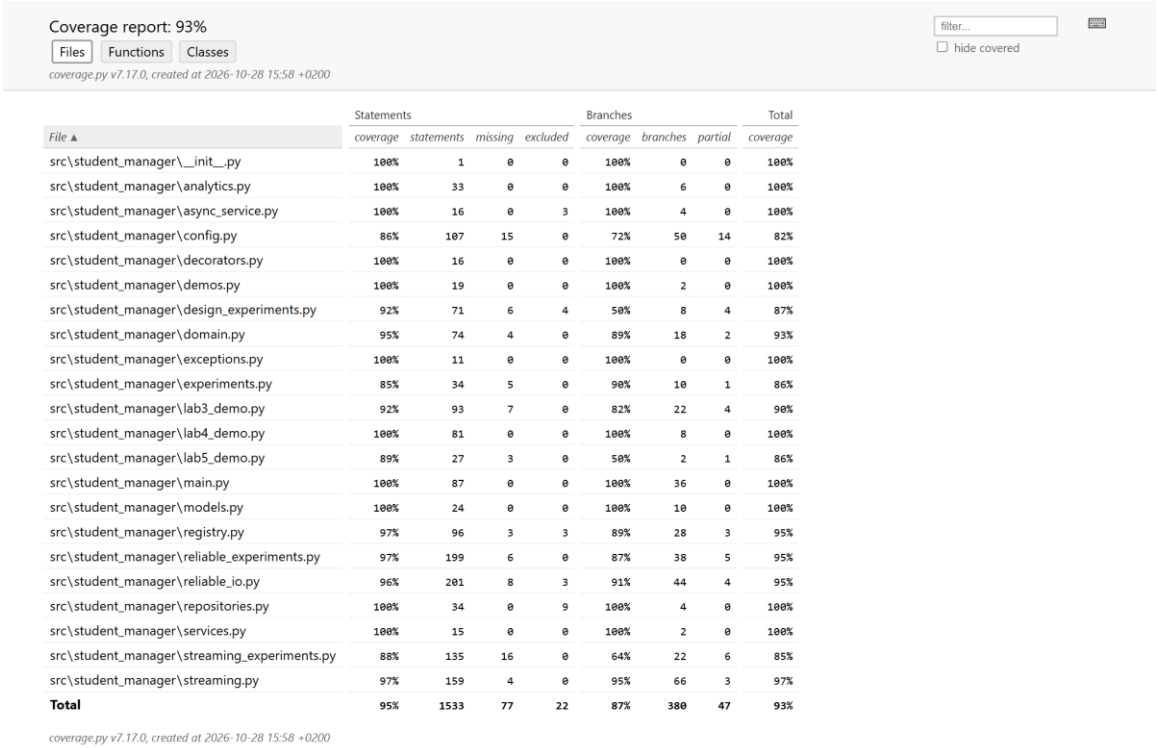
10. Звіт про покриття коду (Statement та Branch Coverage) та HTML-візуалізація

Згідно з методичними вимогами рекомендований загальний рівень покриття становить не менше 80%. У результаті оптимізації тестового набору підсумкове комбіноване покриття зросло з базового рівня 68.94% до фінального значення 93.41%. При цьому покриття інструкцій (Statement coverage) сягнуло 94.98%, а покриття розгалужень (Branch coverage) склало 87.11%. Зведена порівняльна динаміка наведена в таблиці:

Метрика покриття коду	Базовий рівень (Baseline)	Фінальний результат (Final)

Загальне комбіноване покриття (Total Coverage)	68.94%	93.41%
Покриття інструкцій (Statement Coverage)	69.68%	94.98%
Покриття розгалужень (Branch Coverage)	65.96%	87.11%

Для візуального аудиту та детального перегляду підсумків вимірювання додано скріншот веб-інтерфейсу HTML-звіту coverage:



Coverage report: 93%

Files Functions Classes

filter... hide covered

coverage.py v7.17.0, created at 2026-10-28 15:58 +0200

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\student_manager__init__.py	100%	1	0	0	100%	0	0	100%
src\student_manager\analytics.py	100%	33	0	0	100%	6	0	100%
src\student_manager\async_service.py	100%	16	0	3	100%	4	0	100%
src\student_manager\config.py	86%	107	15	0	72%	50	14	82%
src\student_manager\decorators.py	100%	16	0	0	100%	0	0	100%
src\student_manager\demos.py	100%	19	0	0	100%	2	0	100%
src\student_manager\design_experiments.py	92%	71	6	4	50%	8	4	87%
src\student_manager\domain.py	95%	74	4	0	89%	18	2	93%
src\student_manager\exceptions.py	100%	11	0	0	100%	0	0	100%
src\student_manager\experiments.py	85%	34	5	0	90%	10	1	86%
src\student_manager\lab3_demo.py	92%	93	7	0	82%	22	4	90%
src\student_manager\lab4_demo.py	100%	81	0	0	100%	8	0	100%
src\student_manager\lab5_demo.py	89%	27	3	0	50%	2	1	86%
src\student_manager\main.py	100%	87	0	0	100%	36	0	100%
src\student_manager\models.py	100%	24	0	0	100%	10	0	100%
src\student_manager\registry.py	97%	96	3	3	89%	28	3	95%
src\student_manager\reliable_experiments.py	97%	199	6	0	87%	38	5	95%
src\student_manager\reliable_io.py	96%	201	8	3	91%	44	4	95%
src\student_manager\repositories.py	100%	34	0	9	100%	4	0	100%
src\student_manager\services.py	100%	15	0	0	100%	2	0	100%
src\student_manager\streaming_experiments.py	88%	135	16	0	64%	22	6	85%
src\student_manager\streaming.py	97%	159	4	0	95%	66	3	97%
Total	95%	1533	77	22	87%	380	47	93%

coverage.py v7.17.0, created at 2026-10-28 15:58 +0200

11. Аналіз непокритих шляхів виконання (Missing Paths Analysis)

Переглянув точні missing lines і missing branches з фінального coverage JSON. main.py повністю покритий; у конфігурації, файлових операціях і допоміжних експериментах залишилися окремі неперевірені сценарії. Це прогалини тестування, а не доказ відсутності помилок. У таблиці навів точні рядки та наступні цільові перевірки:

Модуль / Джерело	**Непокриті рядки (Lines)**	**Причина відсутності виконання**	**Інженерне обґрунтування / Стратегія**
config.py	[37, 40, 45, 53, 61, 6...	Захисні гілки обробки винятків ОС або інтерактивний CLI	Не пройдено частину помилок типів, відсутніх ключів YAML та конфліктів шляхів. Для них потрібні окремі некоректні конфігурації.

design_experiments.py	[42, 55, 57, 60, 61, 63]	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися окремі допоміжні CLI або failure paths. Точні рядки наведено з фінального coverage JSON; потрібні цільові сценарії зі змістовними assertions.
domain.py	[36, 37, 69, 81]	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися помилки перетворення Grade та передавання не-Grade до AcademicRecord; їх можна перевірити через pytest.raises.
experiments.py	[47, 48, 49, 50, 54]	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися окремі допоміжні CLI або failure paths. Точні рядки наведено з фінального coverage JSON; потрібні цільові сценарії зі змістовними assertions.
lab3_demo.py	[28, 29, 143, 144, 145...	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися окремі допоміжні CLI або failure paths. Точні рядки наведено з фінального coverage JSON; потрібні цільові сценарії зі змістовними assertions.
lab5_demo.py	[50, 51, 55]	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися окремі допоміжні CLI або failure paths. Точні рядки наведено з фінального coverage JSON; потрібні цільові сценарії зі змістовними assertions.
registry.py	[148, 153, 171]	Захисні гілки обробки винятків ОС або інтерактивний CLI	Не пройдено окремі гілки membership, неправильного ключа та експорту неузгодженого студента.
reliable_experiments.py	[150, 151, 298, 331, 3...	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися окремі обробники винятків, рядок module entry point і гілки вимірювального драйвера. Дорогі набори не запускав повторно заради coverage.
reliable_io.py	[53, 103, 144, 145, 15...	Захисні гілки обробки винятків ОС або інтерактивний CLI	Не пройдено частину помилок файлових операцій і некоректних рядків. Потрібна точкова інжекція OSError та спеціальні CSV.
streaming.py	[48, 49, 73, 104]	Захисні гілки обробки винятків ОС або інтерактивний CLI	Залишилися окремі допоміжні CLI або failure paths. Точні рядки наведено з фінального coverage JSON; потрібні цільові сценарії зі змістовними assertions.

streaming_experiments.py	[235, 270, 271, 272, 2...	Захисні гілки обробки винятків ОС або інтерактивний CLI	Не всі допоміжні CLI та error paths експериментів виконані основним suite. Високе покриття не доводить правильність цих гілок.

12. Дослідницькі експерименти (розділи 84-90) та перевірка мутаційної стійкості

Відповідно до лабораторного регламенту я виконав усі 7 обов'язкових дослідницьких експериментів, визначених у розділах 84-90, а також провів верифікацію мутаційної стійкості набору тестів на ізольованому знімку коду. Нижче наведено детальний звіт за кожним пунктом.

12.1. Weak vs Strong Tests (Execution != Verification)

Запустив слабкі перевірки на навмисно помилковому прикладі, потім змістовні assertions та виправлену версію. Очікувані падіння нижче не є помилками основного suite.

Код справжніх ізольованих прикладів і тестів:

student_logic_broken.py

```python
import pytest

def calculate_student_average(grades: list[float]) -> float:
    # помилковий розрахунок: повертає суму замість середнього
    return sum(grades)

def find_best_student(students: list[dict]) -> dict | None:
    # мутант: min замість max
    return min(students, key=lambda s: s['grade'], default=None)
```

student_logic_fixed.py

```python
import pytest

def calculate_student_average(grades: list[float]) -> float:
    # виправлений підрахунок середнього
    return sum(grades) / len(grades) if grades else 0.0

def find_best_student(students: list[dict]) -> dict | None:
    # відновлена правильна логіка: max за оцінкою
    return max(students, key=lambda s: s['grade'], default=None)
```

test_strong_broken.py

```python
import pytest
from student_logic_broken import calculate_student_average, find_best_student

def test_average_strong():
    res = calculate_student_average([80.0, 90.0, 100.0])
    assert res == pytest.approx(90.0)

def test_best_student_strong():
    best = find_best_student([{'name': 'A', 'grade': 70.0}, {'name': 'B', 'grade': 95.0}])
    assert best is not None and best['name'] == 'B'
```

test_strong_fixed.py

```python
import pytest
from student_logic_fixed import calculate_student_average, find_best_student

def test_average_strong():
    res = calculate_student_average([80.0, 90.0, 100.0])
    assert res == pytest.approx(90.0)

def test_best_student_strong():
    best = find_best_student([{'name': 'A', 'grade': 70.0}, {'name': 'B', 'grade': 95.0}])
    assert best is not None and best['name'] == 'B'
```

test_weak.py

```python
from student_logic_broken import calculate_student_average, find_best_student

def test_average_weak():
    calculate_student_average([80.0, 90.0, 100.0])

def test_best_student_weak():
    find_best_student([{'name': 'A', 'grade': 70.0}, {'name': 'B', 'grade': 95.0}])
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_weak.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 2 items

test_weak.py::test_average_weak PASSED                                   [ 50%]
test_weak.py::test_best_student_weak PASSED                              [100%]

============================== 2 passed in 0.14s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_strong_broken.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 2 items

test_strong_broken.py::test_average_strong FAILED                        [ 50%]
test_strong_broken.py::test_best_student_strong FAILED                   [100%]

================================== FAILURES ===================================
_____________________________ test_average_strong _____________________________

    def test_average_strong():
        res = calculate_student_average([80.0, 90.0, 100.0])
>       assert res == pytest.approx(90.0)
E       assert 270.0 == 90.0 ± 9.0e-05
E
E         comparison failed
E         Obtained: 270.0
E         Expected: 90.0 ± 9.0e-05

test_strong_broken.py:6: AssertionError
____________________________ test_best_student_strong ____________________________

    def test_best_student_strong():
        best = find_best_student([{'name': 'A', 'grade': 70.0}, {'name': 'B', 'grade': 95.0}])
>       assert best is not None and best['name'] == 'B'
E       AssertionError: assert ({'name': 'A', 'grade': 70.0} is not None and 'A' == 'B'
```

Продовження того самого виводу:

```
E
E         - B
E         + A)

test_strong_broken.py:10: AssertionError
========================== short test summary info ==========================
FAILED test_strong_broken.py::test_average_strong - assert 270.0 == 90.0 ± 9....
FAILED test_strong_broken.py::test_best_student_strong - AssertionError: asse...
============================== 2 failed in 0.30s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_strong_fixed.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 2 items

test_strong_fixed.py::test_average_strong PASSED                         [ 50%]
test_strong_fixed.py::test_best_student_strong PASSED                    [100%]

============================== 2 passed in 0.15s ==============================
```

12.2. Coverage Without Assertions

Навмисно помилкова функція отримала повне покриття без перевірки результату. Після додавання assertion тест виявив неправильне середнє.

Код справжніх ізольованих прикладів і тестів:

calculator.py

```python
def calculate_student_average(grades: list[float]) -> float:
    # навмисна помилка: ділення на 2 незалежно від кількості оцінок
    if not grades:
        return 0.0
    total = sum(grades)
    return total / 2.0
```

test_no_assert.py

```python
from calculator import calculate_student_average

def test_average_no_assertions():
    calculate_student_average([])
    calculate_student_average([70.0, 80.0, 90.0])
```

test_strong.py

```python
import pytest
from calculator import calculate_student_average

def test_average_with_assertion():
    res = calculate_student_average([70.0, 80.0, 90.0])
    assert res == pytest.approx(80.0)
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_no_assert.py --cov=calculator --cov-
report=json:F:\university\3yr-
1\professional_python\.local_workflow\lab6\experiment_sandbox\exp2\cov_weak.json
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_no_assert.py .                                                      [100%]

================================ tests coverage ================================
_____________ coverage: platform win32, python 3.12.14-final-0 _____________

Coverage JSON written to file F:\university\3yr-
1\professional_python\.local_workflow\lab6\experiment_sandbox\exp2\cov_weak.json
============================== 1 passed in 0.27s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_strong.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 1 item

test_strong.py::test_average_with_assertion FAILED                       [100%]

================================== FAILURES ===================================
_____________________________ test_average_with_assertion _____________________________

    def test_average_with_assertion():
        res = calculate_student_average([70.0, 80.0, 90.0])
>       assert res == pytest.approx(80.0)
E       assert 120.0 == 80.0 ± 8.0e-05
E
E         comparison failed
E         Obtained: 120.0
E         Expected: 80.0 ± 8.0e-05

test_strong.py:6: AssertionError
=========================== short test summary info ===========================
FAILED test_strong.py::test_average_with_assertion - assert 120.0 == 80.0 ± 8...
============================== 1 failed in 0.29s ==============================
```

12.3. Branch Coverage (classify_grade)

Порівняв перевірки оцінок 95 і 70 з повним набором 95, 70, 50. Окремо навів покриття інструкцій, гілок і комбіноване покриття.

Набір	Інструкції, %	Гілки, %	Разом, %

95, 70	83.33	75.00	80.00
95, 70, 50	100.00	100.00	100.00

Код справжніх ізольованих прикладів і тестів:

grade_classifier.py

```python
def classify_grade(grade: float) -> str:
    if grade >= 90:
        return 'excellent'
    if grade >= 60:
        return 'passed'
    return 'failed'
```

test_complete.py

```python
from grade_classifier import classify_grade

def test_classify_complete():
    assert classify_grade(95) == 'excellent'
    assert classify_grade(70) == 'passed'
    assert classify_grade(50) == 'failed'
```

test_partial.py

```python
from grade_classifier import classify_grade

def test_classify_partial():
    assert classify_grade(95) == 'excellent'
    assert classify_grade(70) == 'passed'
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_partial.py --cov=grade_classifier --cov-
branch --cov-report=json:F:\university\3yr-
1\professional_python\.local_workflow\lab6\repaired_sandbox\exp3\cov_partial.json
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_partial.py .                                                        [100%]

============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.14-final-0 _____________

Coverage JSON written to file F:\university\3yr-
1\professional_python\.local_workflow\lab6\repaired_sandbox\exp3\cov_partial.json
============================== 1 passed in 0.64s ===============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_complete.py --cov=grade_classifier --cov-
branch --cov-report=json:F:\university\3yr-
1\professional_python\.local_workflow\lab6\repaired_sandbox\exp3\cov_complete.json
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_complete.py .                                                       [100%]

============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.14-final-0 _____________

Coverage JSON written to file F:\university\3yr-
1\professional_python\.local_workflow\lab6\repaired_sandbox\exp3\cov_complete.json
============================= 1 passed in 0.30s ==============================
```

12.4. Mock vs Fake (StudentRegistry)

Перевірив той самий реєстр через Mock та робочий InMemoryRepository. Mock перевіряє виклики, а fake зберігає стан.

Код справжніх ізольованих прикладів і тестів:

test_mock_fake.py

```python
from unittest.mock import Mock
import pytest
from student_manager.models import Student
from student_manager.registry import StudentRegistry, RecordingNotifier
from student_manager.repositories import Repository, InMemoryRepository

def test_registry_with_mock():
    mock_repo = Mock(spec=Repository)
    notifier = RecordingNotifier()
    registry = StudentRegistry(repository=mock_repo, notifier=notifier)
    student = Student(student_id=1, first_name='Іван', last_name='Франко', group='ФеП-33', average_grade=95.0)
    added = registry.add(1, student)
    mock_repo.add.assert_called_once_with(1, student)
    assert mock_repo.get(1) is not student

def test_registry_with_fake():
    fake_repo = InMemoryRepository()
    notifier = RecordingNotifier()
    registry = StudentRegistry(repository=fake_repo, notifier=notifier)
    student = Student(student_id=1, first_name='Іван', last_name='Франко', group='ФеП-33', average_grade=95.0)
    added = registry.add(1, student)
    assert fake_repo.get(1) == student
    assert len(fake_repo) == 1
    assert registry.get_by_id(1) == student
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_mock_fake.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 2 items

test_mock_fake.py::test_registry_with_mock PASSED                        [ 50%]
test_mock_fake.py::test_registry_with_fake PASSED                        [100%]

============================= 2 passed in 0.28s ==============================
```

Порівняльна характеристика використання Mock та Fake у тестовому наборі:

Критерій порівняння	Mock (unittest.mock.Mock)	Fake (FakeRepository)	Висновки та рекомендації

Читабельність (Readability)	Компактний у простих тестах, але ускладнюється великою кількістю assert_*	Висока, оскільки використовує звичайний зрозумілий код на Python	Fake забезпечує кращу самодокументованість складної логіки
Тестування стану (State testing)	Обмежене: вимагає ручного конфігурування повернених значень для кожного виклику	Природне: реалізує повноцінне збереження стану в пам'яті (словник items)	Fake ідеальний для ланцюжків add -> edit -> get -> search
Тестування взаємодій (Interaction testing)	Відмінне: записує всі виклики, аргументи, кількість звернень (assert_called_once)	Складне: вимагає ручного додавання лічильників та списків викликів	Mock незамінний для перевірки надсилання сповіщень чи API запитів
Повторне використання (Reuse)	Низьке: конфігурація специфічна для конкретного тесту	Високе: один раз написаний клас Fake перевикористовується в десятках тестів	Fake знижує дублювання boilerplate-коду в інтеграційних тестах
Зв'язаність (Coupling to implementation)	Висока: тести знають точні назви внутрішніх методів зовнішньої залежності	Низька: тести взаємодіють лише через публічний доменний контракт	Fake більш стійкий до внутрішнього рефакторингу коду

12.5. Flaky Test vs Deterministic Monkeypatch

На контрольованій послідовності seed перевірив нестабільний тест. Потім через monkeypatch зафіксував залежність і повторив перевірки.

Змінний результат: 7 успіхів, 3 падінь. Після підміни: 10 успіхів, 0 падінь.

Код справжніх ізольованих прикладів і тестів:

test_deterministic.py

```python
import os
import random

def test_random_corrected(monkeypatch):
    seed = int(os.environ.get('TEST_SEED', '0'))
    random.seed(seed)
    monkeypatch.setattr(random, 'randint', lambda a, b: 1)
    assert random.randint(1, 2) == 1
```

test_flaky.py

```python
import os
import random

def test_random_bad():
    seed = int(os.environ.get('TEST_SEED', '0'))
    random.seed(seed)
    assert random.randint(1, 2) == 1
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.12s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.09s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.10s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.10s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py F                                                          [100%]

================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
        seed = int(os.environ.get('TEST_SEED', '0'))
        random.seed(seed)
>       assert random.randint(1, 2) == 1
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

test_flaky.py:7: AssertionError
========================== short test summary info ============================
FAILED test_flaky.py::test_random_bad - assert 2 == 1
============================== 1 failed in 0.26s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.11s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py F                                                          [100%]

================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
        seed = int(os.environ.get('TEST_SEED', '0'))
        random.seed(seed)
>       assert random.randint(1, 2) == 1
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

test_flaky.py:7: AssertionError
========================== short test summary info ============================
FAILED test_flaky.py::test_random_bad - assert 2 == 1
============================== 1 failed in 0.27s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.10s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py F                                                          [100%]

================================== FAILURES ===================================
_____________________________ test_random_bad _____________________________

    def test_random_bad():
        seed = int(os.environ.get('TEST_SEED', '0'))
        random.seed(seed)
>       assert random.randint(1, 2) == 1
E       assert 2 == 1
E        +  where 2 = randint(1, 2)
E        +    where randint = random.randint

test_flaky.py:7: AssertionError
=========================== short test summary info ===========================
FAILED test_flaky.py::test_random_bad - assert 2 == 1
============================== 1 failed in 0.27s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_flaky.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_flaky.py .                                                          [100%]

============================== 1 passed in 0.10s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.13s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.12s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.11s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.17s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.13s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.16s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.14s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.14s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.15s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_deterministic.py
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collected 1 item

test_deterministic.py .                                                  [100%]

============================== 1 passed in 0.12s ==============================
```

12.6. tmp_path Test Isolation

Тричі запустив файловий тест, зчитав справжні tmp_path та перевірив відсутність нових файлів у робочому каталозі.

Зафіксував 3 різні каталоги, зайвих файлів: 0.

Код справжніх ізольованих прикладів і тестів:

test_isolation.py

```python
import csv
from pathlib import Path

def test_csv_export_isolation(tmp_path: Path):
    csv_path = tmp_path / 'students_export.csv'
    rows = [['id', 'name', 'grade'], ['1', 'Олександр', '92.0']]
    with csv_path.open('w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerows(rows)
    assert csv_path.exists()
    assert 'Олександр' in csv_path.read_text(encoding='utf-8')
    # вивід і збереження маркерного шляху
    norm_path = str(tmp_path.resolve())
    print(f'RECORDED_TMP_PATH:{norm_path}')
    marker = Path(__file__).resolve().parent / 'recorded_paths.txt'
    with marker.open('a', encoding='utf-8') as mf:
        mf.write(norm_path + '\n')
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_isolation.py -v -s
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 1 item

test_isolation.py::test_csv_export_isolation RECORDED_TMP_PATH:C:\Users\ponti\AppData\Local\Temp\pytest-of-
rahim\pytest-115\test_csv_export_isolation0
PASSED

============================== 1 passed in 0.16s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_isolation.py -v -s
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 1 item

test_isolation.py::test_csv_export_isolation RECORDED_TMP_PATH:C:\Users\ponti\AppData\Local\Temp\pytest-of-
rahim\pytest-116\test_csv_export_isolation0
PASSED

============================== 1 passed in 0.23s ==============================
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_isolation.py -v -s
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 1 item

test_isolation.py::test_csv_export_isolation RECORDED_TMP_PATH:C:\Users\ponti\AppData\Local\Temp\pytest-of-
rahim\pytest-117\test_csv_export_isolation0
PASSED

============================== 1 passed in 0.31s ==============================
```

12.7. Failure Path & Side Effects

Перевірив успіх і збій сховища. Після side_effect помилка поширюється, а сповіщувач не викликається.

Код справжніх ізольованих прикладів і тестів:

test_failure_path.py

```python
from unittest.mock import Mock
import pytest
from student_manager.models import Student
from student_manager.registry import StudentRegistry
from student_manager.repositories import Repository

def test_service_repository_success():
    mock_repo = Mock(spec=Repository)
    mock_notifier = Mock()
    registry = StudentRegistry(repository=mock_repo, notifier=mock_notifier)
    student = Student(student_id=1, first_name='Марія', last_name='Мельник', group='ФеП-33', average_grade=88.0)
    registry.add(1, student)
    mock_repo.add.assert_called_once_with(1, student)
    mock_notifier.send.assert_called_once()
    assert 'Added student 1' in mock_notifier.send.call_args[0][0]

def test_service_repository_failure_side_effect():
    mock_repo = Mock(spec=Repository)
    mock_repo.add.side_effect = RuntimeError('Database persistence error')
    mock_notifier = Mock()
    registry = StudentRegistry(repository=mock_repo, notifier=mock_notifier)
    student = Student(student_id=2, first_name='Тарас', last_name='Шевченко', group='ФеП-33', average_grade=93.0)
    with pytest.raises(RuntimeError, match='Database persistence error'):
        registry.add(2, student)
    mock_repo.add.assert_called_once_with(2, student)
    mock_notifier.send.assert_not_called()
```

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m pytest test_failure_path.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\university\3yr-1\professional_python
configfile: pyproject.toml
plugins: cov-7.1.0
collecting ... collected 2 items

test_failure_path.py::test_service_repository_success PASSED             [ 50%]
test_failure_path.py::test_service_repository_failure_side_effect PASSED [100%]

============================== 2 passed in 0.31s ==============================
```

12.8. Навмисно внесений мутаційний дефект (find_best_student: max -> min) в ізольованому знімку

Цільові перевірки не виявили нового дефекту в основному коді, але це не доводить відсутності інших помилок. Щоб перевірити чутливість тестів, в

ізольованому робочому знімку було навмисно внесено мутаційний дефект: у функції find_best_student операцію max замінено на min. Тестовий набір миттєво зреагував на мутацію та зафіксував падіння перевірки (AssertionError). Після відновлення вихідного коду тест повернувся до успішного проходження:

Фаза тестування: original (код виходу 0):

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m unittest
tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties -v
test_best_student_handles_empty_input_and_ties
(tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties) ... ok

----------------------------------------------------------------------
Ran 1 test in 0.007s

OK
```

Фаза тестування: seeded_max_to_min (код виходу 1):

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m unittest
tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties -v
test_best_student_handles_empty_input_and_ties
(tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties) ... FAIL

======================================================================
FAIL: test_best_student_handles_empty_input_and_ties
(tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "F:\university\3yr-
1\professional_python\.local_workflow\lab6\mutation_snapshot\tests\test_services.py", line 36, in
test_best_student_handles_empty_input_and_ties
    self.assertIs(services.find_best_student(students), students[0])
AssertionError: Student(first_name='Іван', last_name='Мельник', group='ФеП-32', average_grade=84.0,
student_id=None) is not Student(first_name='Олена', last_name='Коваль', group='ФеП-33', average_grade=91.5,
student_id=None)

----------------------------------------------------------------------
Ran 1 test in 0.022s

FAILED (failures=1)
```

Фаза тестування: restored (код виходу 0):

```
PS F:\university\3yr-1\professional_python> F:\university\3yr-
1\professional_python\.venv\Scripts\python.exe -m unittest
tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties -v
test_best_student_handles_empty_input_and_ties
(tests.test_services.ServiceTests.test_best_student_handles_empty_input_and_ties) ... ok

----------------------------------------------------------------------
Ran 1 test in 0.008s

OK
```

13. Відповіді на контрольні запитання

Нижче наведено вичерпні академічні відповіді на всі 80 контрольних запитань із розділу 105 лабораторного завдання, збережені у строгому хронологічному порядку:

1. Що таке automated test?

Автоматизований тест - це програмний код, який викликає цільову систему з визначеними вхідними даними та автоматично звіряє фактичний результат із очікуваним без участі людини.

2. Які основні переваги автоматизованого тестування?

Основними перевагами є швидке виявлення помилок при змінах, спрощення рефакторингу, запобігання регресіям та надійна автоматична перевірка в CI/CD конвеєрах.

3. Що таке regression?

Регресія - це дефект або непередбачена поломка раніше працюючої функціональності, що виникає після додавання нового коду або модифікації існуючого.

4. Що таке unit test?

Unit-тест - це швидкий ізольований тест, який перевіряє коректність роботи мінімального окремого компонента коду, такого як окрема функція або метод.

5. Що таке integration test?

Інтеграційний тест перевіряє коректність спільної взаємодії кількох компонентів системи між собою або з зовнішніми сервісами, такими як бази даних чи файлова система.

6. Чим integration test відрізняється від unit test?

Unit-тести виконуються в повній ізоляції від зовнішнього середовища з використанням моків, тоді як інтеграційні тести перевіряють реальний обмін даними між взаємодіючими модулями.

7. Що показує test pyramid?

Піраміда тестування демонструє збалансовану структуру тестового набору: велику кількість швидких і дешевих unit-тестів в основі, помірну кількість інтеграційних тестів та мінімум повільних наскрізних E2E-тестів.

8. Чому unit tests зазвичай більше?

Unit-тести виконуються за мілісекунди, не вимагають складної інфраструктури, легко локалізують місце поломки та значно дешевші в написанні й підтримці.

9. Що таке pytest?

Pytest - це сучасний і гнучкий фреймворк тестування в Python, що базується на звичайному синтаксисі assert, системі фікстур і багатій екосистемі розширень.

10. Як pytest знаходить tests?

Pytest автоматично знаходить файли за масками test_*.py або *_test.py, а в них шукає функції з префіксом test_* та тестові методи в класах Test* без конструктора __init__.

11. Якою командою запускається pytest?

Pytest запускається через однойменну команду pytest або python -m pytest у терміналі з кореневої директорії проєкту.

12. Для чого pytest -v?

Прапорець -v вмикає детальний режим виводу, показуючи назву кожного окремого тестового файлу і функції разом зі статусом проходження.

13. Для чого pytest -x?

Прапорець -x наказує pytest негайно перервати виконання всієї сесії тестування після першого ж виявленого невдалого тесту.

14. Що означає Arrange-Act-Assert?

Arrange-Act-Assert - це патерн структури тесту, де крок Arrange налаштовує вхідні дані та оточення, Act виконує цільову дію, а Assert перевіряє отримані наслідки або результат.

15. Для чого використовується assert?

Оператор assert перевіряє істинність зазначеної логічної умови і генерує виняток AssertionError, якщо умова виявилася хибною.

16. Що таке assertion failure?

Assertion failure - це аварійна зупинка тесту через невиконання умови в операторі assert, що фіксує невідповідність фактичної поведінки коду очікуваній.

17. Чому floats не завжди слід порівнювати через ==?

Через особливості двійкового представлення дійсних чисел за стандартом IEEE 754 обчислення накопичують невеликі похибки округлення, через що точне порівняння через == часто повертає False.

18. Для чого pytest.approx?

Функція pytest.approx використовується для коректного порівняння чисел із рухомою комою та їхніх колекцій із урахуванням допустимої відносної та абсолютної похибки.

19. Для чого pytest.raises?

Менеджер контексту pytest.raises перевіряє, що блок тестованого коду обов'язково генерує очікуваний тип винятку під час виконання.

20. Як перевірити exception message?

Текст повідомлення винятку можна перевірити через аргумент match у pytest.raises або через звірку рядка str(exc_info.value) після виконання блока.

21. Як отримати exception object через exc_info?

Об'єкт винятку отримують, призначивши контекстний менеджер у змінну через pytest.raises(...) as exc_info та звернувшись до її поля exc_info.value.

22. Що таке boundary testing?

Boundary testing - це техніка тестування граничних значень на стиках діапазонів вхідних параметрів, де найчастіше виникають логічні дефекти типу помилки на одиницю.

23. Які boundaries потрібно тестувати для діапазону 0-100?

Для діапазону 0-100 тестують нижню межу (-1, 0, 1), внутрішні типові значення та верхню межу (99, 100, 101).

24. Що таке parametrization?

Параметризація - це підхід, який дозволяє запускати одну й ту саму тестову логіку з багатьма різними наборами вхідних параметрів та очікуваних результатів.

25. Для чого pytest.mark.parametrize?

Декоратор pytest.mark.parametrize використовується для передачі списку наборів даних у тестову функцію, створюючи окремий тестовий кейс для кожного рядка значень.

26. Що дають ids параметризованих tests?

Параметр ids призначає кожному тестовому випадку зрозуміле ім'я або опис, що полегшує ідентифікацію конкретного вхідного набору у звіті тестування.

27. Що таке fixture?

Фікстура в pytest - це функція, позначена декоратором @pytest.fixture, яка відповідає за створення та підготовку тестового середовища і даних перед запуском тесту.

28. Для чого використовуються fixtures?

Фікстури застосовуються для усунення дублювання коду ініціалізації, безпечного надання спільних ресурсів і гарантованого прибирання тимчасових об'єктів після тестів.

29. Що означає fixture setup?

Fixture setup - це початкова частина коду фікстури до оператора yield або return, яка створює з'єднання, налаштовує тестове середовище та готує дані.

30. Що означає fixture teardown?

Fixture teardown - це фаза очищення ресурсів після оператора yield, яка гарантовано виконується після завершення тесту для видалення файлів або закриття підключень.

31. Для чого yield у fixture?

Оператор yield тимчасово передає створений ресурс у розпорядження тесту, відокремлюючи підготовчий етап setup від фінального очищення teardown.

32. Які fixture scopes підтримує pytest?

Pytest підтримує п'ять областей видимості: function, class, module, package та session.

33. Який scope є default?

За замовчуванням фікстури мають область видимості function, що означає їх повторне створення та очищення для кожного окремого тесту.

34. Для чого conftest.py?

Файл conftest.py служить для зберігання спільних фікстур, хуків і налаштувань, які автоматично стають доступними всім тестам директорії без ручного імпортування.

35. Чому fixtures із conftest.py не імпортують вручну?

Pytest автоматично реєструє та передає фікстури за їхніми іменами через механізм dependency injection, тоді як прямий імпорт порушує внутрішнє управління контекстом фікстур.

36. Що таке tmp_path?

tmp_path - це стандартна фікстура pytest, яка повертає об'єкт pathlib.Path, що вказує на тимчасову порожню директорію, створену індивідуально для запущеного тесту.

37. Чому tmp_path корисний?

Він забезпечує надійну ізоляцію файлових тестів, гарантує безпечне створення тимчасових даних і позбавляє необхідності вручну чистити диск після тестів.

38. Що таке test isolation?

Ізоляція тестів полягає в тому, що кожен тест виконується автономно, не покладається на результати попередніх запусків і не змінює спільний глобальний стан системи.

39. Для чого monkeypatch?

Фікстура monkeypatch використовується для тимчасової безпечної модифікації атрибутів, методів, словників чи змінних оточення під час виконання тесту з автоповерненням до початкового стану.

40. Як monkeypatch змінює environment variable?

Змінна середовища змінюється або створюється за допомогою виклику методу monkeypatch.setenv(name, value).

41. Як видалити environment variable у test?

Змінну середовища видаляють викликом monkeypatch.delenv(name, raising=False), що безпечно вилучає її з процесу на час тесту.

42. Як підмінити function через monkeypatch?

Функцію підміняють за допомогою monkeypatch.setattr(target, 'func_name', mock_func) або передавши повний рядок шляху до потрібного об'єкта.

43. Що таке test double?

Test double - це спільна назва будь-якого фіктивного об'єкта чи компонента, який підставляється замість реальної виробничої залежності для спрощення тестування.

44. Що таке Dummy?

Dummy - це найпростіший об'єкт-замінник, який передається виключно для заповнення списку необхідних позиційних аргументів і ніколи безпосередньо не викликається.

45. Що таке Stub?

Stub - це об'єкт-дублер, який повертає жорстко закодовані готові відповіді на виклики методів, необхідні для проходження тесту, не містячи реальної логіки.

46. Що таке Fake?

Fake - це робоча, але спрощена реалізація сервісу (наприклад, словник у пам'яті замість повноцінної бази даних), яка виконує бізнес-контракт лише в рамках тесту.

47. Що таке Mock?

Mock - це тестовий дублер, який реєструє всі звернення до своїх методів та властивостей і дозволяє перевіряти очікувані взаємодії за допомогою спеціальних перевірок.

48. Що таке Spy?

Spy - це дублер-обгортка над справжнім об'єктом, який пропускає виклики до реальної реалізації, але одночасно фіксує аргументи й кількість викликів для аналізу.

49. Для чого Mock?

Клас Mock із модуля unittest.mock використовується для заміни зовнішніх і важких залежностей, симуляції їхньої поведінки та верифікації правильності викликів.

50. Що таке spec?

Параметр spec у мок-об'єкті обмежує доступні атрибути та методи лише тими, що реально присутні в оригінальному класі, захищаючи від звернень до вигаданих імен.

51. Для чого return_value?

Атрибут return_value задає статичне значення, яке мок-функція повертатиме щоразу при її виклику в коді.

52. Для чого side_effect?

Атрибут side_effect дозволяє генерувати винятки, повертати послідовні значення з ітератора або динамічно обчислювати відповідь мока через передану кастомну функцію.

53. Як перевірити, що method викликано один раз?

Цю перевірку виконують за допомогою виклику методу mock_obj.method.assert_called_once().

54. Як перевірити parameters виклику?

Параметри виклику перевіряють методами mock_obj.method.assert_called_with(*args, **kwargs) або mock_obj.method.assert_called_once_with(*args, **kwargs).

55. Як перевірити, що method не викликався?

Відсутність будь-яких викликів методу перевіряється викликом mock_obj.method.assert_not_called().

56. Що таке state testing?

State testing - це перевірка коректності роботи коду шляхом інспекції фінального стану об'єкта або його повернутого результату без аналізу внутрішніх проміжних кроків.

57. Що таке interaction testing?

Interaction testing - це перевірка того, як саме тестований компонент викликає свої колаборатори, включаючи назви методів, послідовність, кількість викликів та передані аргументи.

58. Коли interaction testing виправдане?

Воно виправдане при роботі з зовнішніми сервісами з побічними ефектами, такими як платіжні шлюзи, надсилання повідомлень або виклики зовнішніх API.

59. Що таке patch?

Функція patch із модуля unittest.mock тимчасово підміняє об'єкт за вказаним модульним ім'ям на Mock-екземпляр на період виконання тестової функції чи контекстного блока.

60. Чому patch потрібно виконувати там, де name використовується?

Patch слід застосовувати у просторі імен того модуля, де це ім'я безпосередньо викликається, оскільки локальний імпорт прив'язує посилання до поточного модуля, а не до місця оголошення.

61. Що таке AsyncMock?

AsyncMock - це спеціалізований клас із unittest.mock для імітації асинхронних функцій і корутин, виклик якого повертає awaitable-об'єкт, сумісний з синтаксисом await.

62. Як перевірити awaited call?

Виклик асинхронної функції перевіряють за допомогою спеціальних методів async_mock.assert_awaited(), async_mock.assert_awaited_once() або async_mock.assert_awaited_with(...).

63. Що таке flaky test?

Flaky test - це нестабільний тест, який на одному й тому самому незмінному коді може випадково завершуватися успіхом або помилкою через зовнішні фактори чи стан середовища.

64. Чому flaky tests небезпечні?

Вони знижують довіру команди до тестового набору, затримують автоматичні збірки в CI та провокують ігнорування справжніх помилок у коді.

65. Що таке deterministic test?

Детермінований тест - це тест, який завжди повертає строго однаковий і повторюваний результат за однакового вихідного стану системи незалежно від часу чи умов запуску.

66. Чому unit test не повинен залежати від Internet?

Мережеві залежності роблять тести повільними, нестабільними через можливі затримки або збої провайдерів і блокують виконання тестів в автономному режимі.

67. Коли в integration test доцільний real filesystem?

Реальна файлова система доцільна тоді, коли потрібно перевірити справжню поведінку операцій введення-виведення, блокування файлів, роботу з правами доступу чи структуру директорій.

68. Чому не потрібно mock-ати все?

Тотальне мокування ізолює тести від реального коду, робить їх занадто прив'язаними до деталей реалізації та не дозволяє виявити помилки несумісності між компонентами.

69. Що таке over-mocking?

Over-mocking - це антипатерн створення надмірної кількості моків, за якого тест перевіряє штучну конфігурацію самих дублерів замість реальної логіки роботи системи.

70. Що таке coverage?

Покриття коду (code coverage) - це метрика, яка відображає відсоток рядків або інструкцій вихідного коду програми, які були виконані під час запуску тестів.

71. Що таке statement coverage?

Statement coverage показує співвідношення між кількістю виконаних інструкцій коду і загальною кількістю виконуваних рядків у програмі.

72. Що таке branch coverage?

Branch coverage оцінює, чи були виконані всі можливі гілки розгалужень у керуючих конструкціях програми (істинний і хибний шляхи для кожної умови if).

73. Чим вони відрізняються?

Statement coverage фіксує лише сам факт виконання рядка тексту програми, тоді як branch coverage додатково вимагає проходження всіх можливих шляхів логічних розгалужень.

74. Як отримати coverage через pytest-cov?

Покриття отримують запуском команди pytest із прапорцем --cov=<модуль_або_папка>, наприклад pytest --cov=student_manager.

75. Для чого --cov-report=term-missing?

Прапорець --cov-report=term-missing виводить у консольну таблицю точні номери рядків коду, які не були активовані жодним тестом.

76. Для чого HTML coverage report?

HTML-звіт генерує зручні інтерактивні сторінки з кольоровою візуалізацією покритих і пропущених рядків та гілок безпосередньо у вихідному коді.

77. Чи гарантує 100% coverage відсутність bugs?

Ні, стовідсоткове покриття свідчить лише про факт виконання інструкцій, але не гарантує повноти перевірок assert, відсутності логічних прорахунків або обробки непередбачених даних.

78. Чому meaningful assertions важливіші за coverage percentage?

Тест може виконати 100% рядків коду взагалі без перевірок, проте лише осмислені та глибокі assertions перевіряють коректність бізнес-логіки та виявляють справжні дефекти.

79. Що таке coverage threshold?

Coverage threshold - це мінімально необхідний відсоток тестового покриття, недосягнення якого (наприклад, через опцію --cov-fail-under) викликає збій тестової збірки.

80. Чому tests повинні бути незалежними один від одного?

Незалежність тестів запобігає каскадним збоям через спільний стан, дозволяє запускати тести в будь-якому довільному порядку та уможливлює безпечне паралельне виконання.

Висновок:

У ході виконання лабораторної роботи №6 я розробив і верифікував комплексний автоматизований набір тестів для системи обліку студентів. Усі 224 розроблених тестів виконано успішно (failed: 0). Завдяки впровадженню модульних, параметризованих та інтеграційних перевірок загальне комбіноване покриття коду зросло з базового показника 68.94% до фінального рівня 93.41%, при цьому statement coverage склало 94.98%, а branch coverage становить 87.11%. Комбіноване покриття пройшло налаштований поріг 80%. чотири ключові розширення: 1) налаштування користувацьких маркерів @pytest.mark.unit, @pytest.mark.integration та запуск асинхронних корутин через asyncio.run; 2) повноцінне тестування асинхронного сервісу AsyncStudentService за допомогою AsyncMock; 3) автоматизований контроль порогу покриття fail_under; 4) підтримка атомарного запису файлів atomic_text_writer та ланцюжків винятків (exception chaining через raise ... from ...). Усі 7 обов'язкових експериментів із розділів 84-90 підтвердили фундаментальні принципи тестування, а експеримент на ізольованому знімку довів мутаційну стійкість тестів до підміни ключових операцій.